ALGORITHM SPECIFICATION & ANALYSIS REPORT

Ford-Fulkerson Algorithm: Maximum Network Flow

Residual capacity tracking and augmenting path evaluation for capacitive transportation networks

1. Algorithm Code (Part I: DFS Augmenting Path Method)

Below is Part 1 of the implementation, showing the recursive depth-first search method used to discover available augmenting paths in the residual network:

```
import java.util.*;

public class FordFulkerson {

    static boolean dfs(int[][] residual, int u, int sink,
            boolean[] visited, int[] parent) {

        visited[u] = true;

        if (u == sink) {
            return true;
        }

        for (int v = 0; v < residual.length; v++) {
            if (!visited[v] && residual[u][v] > 0) {
                parent[v] = u;

                if (dfs(residual, v, sink, visited, parent)) {
                    return true;
                }
            }
        }

        return false;
    }
```

Augmenting Path Search: The DFS explores edges where remaining residual capacity `residual[u][v] > 0`. Predecessor links are recorded in `parent[v]` to allow bottleneck capacity backtracking.

1. Algorithm Code (Part II: Max Flow Loop & Main)

Below is Part 2 of the code, containing the residual network augmentation loop, bottleneck updates, and network capacity initialization:

```
    static int maxFlow(int[][] capacity, int source, int sink) {
        int n = capacity.length;
        int[][] residual = new int[n][n];

        for (int i = 0; i < n; i++) {
            residual[i] = capacity[i].clone();
        }

        int[] parent = new int[n];
        int maxFlow = 0;

        while (true) {
            boolean[] visited = new boolean[n];
            Arrays.fill(parent, -1);

            if (!dfs(residual, source, sink, visited, parent)) {
                break;
            }

            int pathFlow = Integer.MAX_VALUE;

            for (int v = sink; v != source; v = parent[v]) {
                int u = parent[v];
                pathFlow = Math.min(pathFlow, residual[u][v]);
            }

            for (int v = sink; v != source; v = parent[v]) {
                int u = parent[v];
                residual[u][v] -= pathFlow;
                residual[v][u] += pathFlow;
            }

            maxFlow += pathFlow;
        }

        return maxFlow;
    }

    public static void main(String[] args) {
        int[][] capacity = {
                { 0, 10, 5, 0 },
                { 0, 0, 0, 10 },
                { 0, 0, 0, 10 },
                { 0, 0, 0, 0 }
        };

        System.out.println(maxFlow(capacity, 0, 3));
    }
}
```

> **Cancellation Invariance:** Adding flow along reverse residual edges allows future iterations to undo previously pushed flow, guaranteeing convergence to the global maximum flow.

2. Output & Step-by-Step Breakdown

```
15
```

The program calculates a maximum throughput of **15 units** across the capacitive network:

- **Initial Network State:** Source Node 0 directs flow to Node 1 (capacity 10) and Node 2 (capacity 5). Nodes 1 and 2 each connect directly to Sink Node 3 with capacity 10.
- **Augmenting Path 1 (0 → 1 → 3):** Bottleneck capacity is `min(10, 10) = 10`. Forward residual edges decrease by 10 (`residual[0][1]=0`, `residual[1][3]=0`), while reverse edges increase. Flow total becomes **10**.
- **Augmenting Path 2 (0 → 2 → 3):** Bottleneck capacity is `min(5, 10) = 5`. Forward residual edges decrease by 5 (`residual[0][2]=0`, `residual[2][3]=5`). Total flow increases to 10 + 5 = **15**.
- **Termination:** In iteration 3, all outgoing edges from source Node 0 have residual capacity 0. DFS returns `false` and the while loop terminates.

3. Detailed Code Walkthrough

Component / Variable	Mechanics & Architectural Function
`residual[][]` matrix	Tracks current forward and backward residual capacities. Initialized as a deep clone of the capacity matrix.
`dfs(...)` traversal	Locates an augmenting path from source to sink in the residual network and populates `parent[]` pointers.
`parent[]` array	Records predecessors to enable backwards path tracing from sink to source for bottleneck calculation.
Bottleneck calculation	Computes `pathFlow = Math.min(...)` along the path, determining the max flow that can be routed.
Residual capacity update	Decrements forward edges by `pathFlow` and increments reverse edges by `pathFlow`.
Loop termination	Breaks when `dfs` returns false, meaning no augmenting path exists with positive residual capacity.

4. Algorithmic Foundation & Working Principle

The algorithm is based on the classical **Max-Flow Min-Cut Theorem** (Ford & Fulkerson, 1956):

- **Max-Flow Min-Cut Equivalence:** The maximum flow from source to sink equals the total capacity of the minimum cut (the smallest sum of edge capacities whose removal disconnects source from sink).
- **Flow Conservation:** For every node other than the source and sink, total incoming flow must equal total outgoing flow, respecting physical link limits.

TIME COMPLEXITY

$O(E \cdot |f^*|)$

Augmenting paths multiplied by max flow value.

AUXILIARY SPACE

$\Theta(V^2)$

Adjacency matrix for capacity and residual flow.

DUALITY PROPERTY

Max Flow = Min Cut

Dual problem identifies the network bottleneck.

5. Practical Uses & Problem Reductions

Network flow models directly solve a diverse range of combinatorial optimization problems:

- **Bipartite Matching:** Formulated as max flow by linking a super-source and super-sink with unit edge capacities.
- **Circulation with Demands:** Modeling multi-source multi-sink distribution graphs under lower and upper consumption bounds.
- **Image Segmentation:** Formulated as min-cut energy minimization to separate foreground from background in computer vision.

6. Real-World Industry Use Cases

Industry Domain	Operational Problem	Ford-Fulkerson Application
Airline Flight Scheduling	Crew assignment and turnaround scheduling.	Solves maximum flight pairings to optimize aircraft fleet utilization and minimize crew layover overhead.
Content Delivery Networks	Backbone traffic routing across global PoPs.	Maximizes video streaming bandwidth allocation across multi-hop fiber routes without congesting physical links.
Oil & Gas Pipelines	Pipeline fluid throughput optimization.	Calculates peak liquid transport capacity from production refineries through distribution pumping networks.
Disaster Logistics	Evacuation route planning during emergencies.	Identifies transportation chokepoints and plans maximal vehicular evacuation throughput during floods or hurricanes.